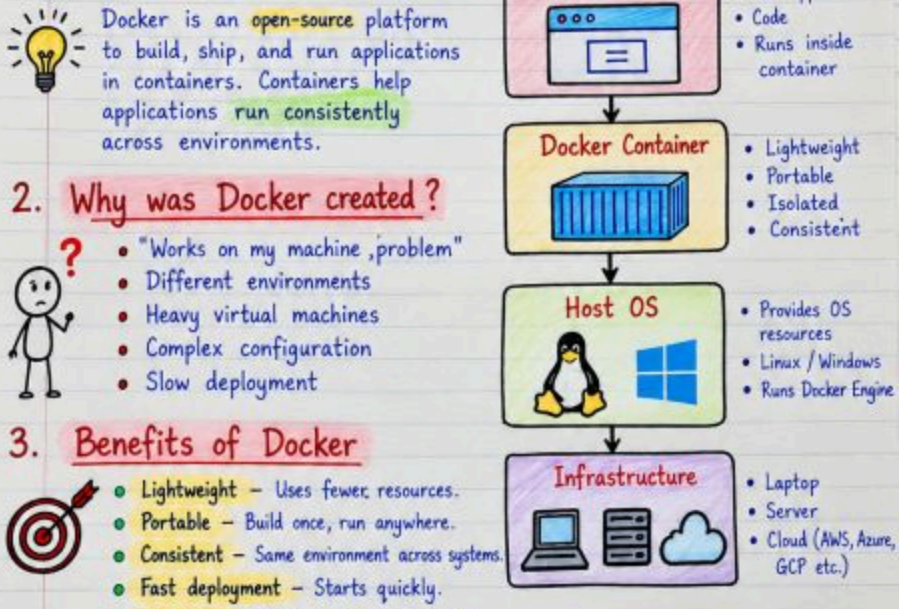

DOCKER
COMPLETE NOTES
Beginner to Advanced
Containers
• Isolated Environment
• Lightweight
• Portability
Images
• Read-only template
• Built using Dockerfile
• Layers
Dockerfile
• Build your own image
• Set instructions
• Define environment
Docker
• Open source platform
• Build, Ship, Run
• Manage containers
Compose
• Run multi-container app
• docker-compose.yml
• Manage services
Networking
• Bridge network
• Host network
• Custom network
Volumes
• Persistent storage
• Share data
• Not lost on container delete
Linux
• Base OS
• Namespaces
• cgroups
Cloud
• Deploy on cloud
• Docker Hub
• ACR, ECR, GCR
Terminal
• Docker CLI
• Important Commands
• Troubleshooting
CI/CD
• Automated build
• Test & Deploy
• Integrate with Jenkins
Kubernetes
• Container Orchestration
• Scale & Manage
• Deploy Containers
What You Will Learn
8 Detailed Notes
Step by Step Examples
Real-time Use Cases
Architecture & Concepts
Interview Ready Topics
Important Docker Commands Cheat Sheet

DOCKER NOTES

1. What is Docker ?

Docker is an open-source platform to build, ship, and run applications in containers. Containers help applications run consistently across environments.



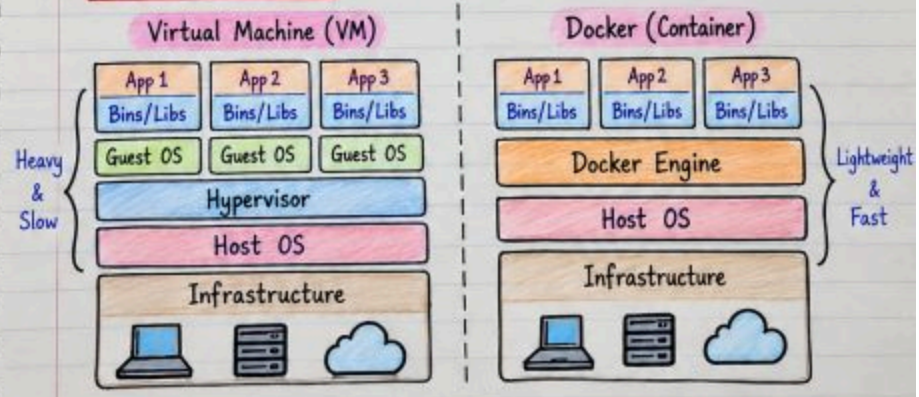
2. Why was Docker created ?

- "Works on my machine ,problem"
- Different environments
- Heavy virtual machines
- Complex configuration
- Slow deployment

3. Benefits of Docker

- **Lightweight** – Uses fewer resources.
- **Portable** – Build once, run anywhere.
- **Consistent** – Same environment across systems.
- **Fast deployment** – Starts quickly.
- **Isolation** – Containers run separately.
- **Scalability** – Scale applications easily.

4. VM vs Docker



DOCKER ARCHITECTURE

Docker follows a Client – Server Architecture

1 Docker Client

- User interacts with Docker using Docker CLI (Command Line Interface) or REST API.
- Sends commands like build, run, pull, push etc. to Docker Daemon.

2 Docker Daemon (dockerd)

- Listens to Docker API requests and manages Docker objects like images, containers, networks and volumes.
- Runs in the background on the host machine.

3 Docker Images

- Read-only templates used to create containers.
- Built using a Dockerfile.
- Stored in Docker Hub or local system.

4 Containers

- Runnable instances of images.
- Lightweight, isolated and portable.
- Each container has its own filesystem, process space and network.

5 Networks

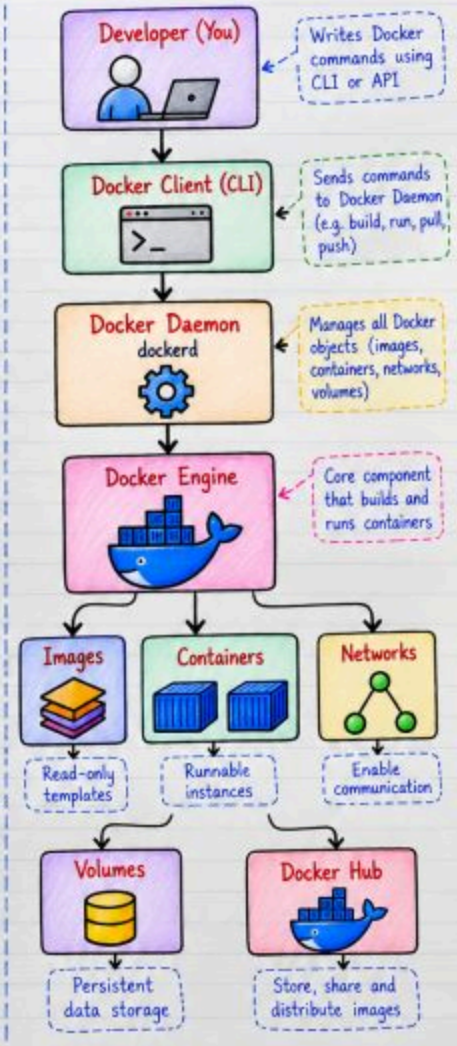
- Enable communication between containers.
- Provide network isolation.
- Types: bridge, host, overlay, macvlan etc.

6 Volumes

- Used to persist data generated by containers even if the container is removed.
- Helps in data sharing between containers.

7 Docker Hub

- A cloud registry service to store, share and distribute Docker images.
- Public and private repositories available.



Key Points

- [x] Docker uses a client-server architecture.
- [x] Docker Daemon (dockerd) is the core of Docker.
- [x] Images are used to create containers.
- [x] Containers are lightweight and portable.
- [x] Networks enable communication between containers.
- [x] Volumes provide persistent data storage.
- [x] Docker Hub stores and distributes images.

Remember

- Build once
- Run anywhere
- Share easily
- Scale as per requirement
- Persistent data using volumes

IMAGE vs CONTAINER vs DOCKERFILE

1. IMAGE

- An image is a **read-only template** that contains the application, dependencies, libraries, tools and other files required to run the app.
- Images are used to create containers.



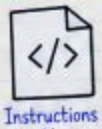
2. CONTAINER

- A container is a **running instance** of an image.
- Containers are lightweight, isolated and portable.
- Each container has its own filesystem, process space and network.



3. DOCKERFILE

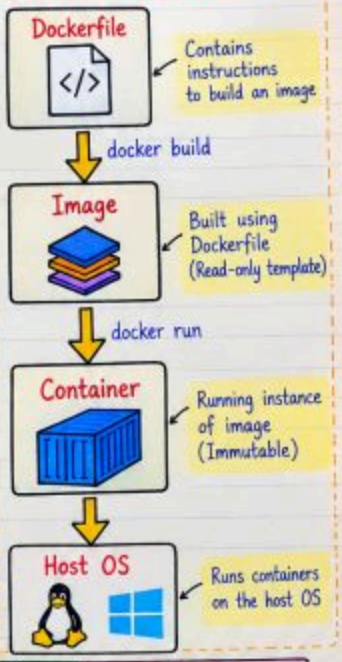
- A Dockerfile is a **text file** that contains a set of instructions to build a Docker image.
- It is used by Docker to automate the image creation process.



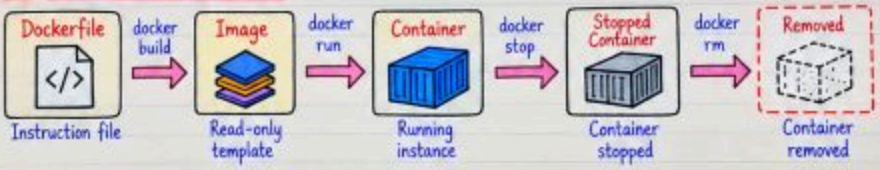
4. COMPARISON TABLE

FEATURE	Image	Container	Dockerfile
Definition	Read-only template	Running instance	Instruction file
State	Static	Dynamic	N/A
Purpose	To create containers	To run applications	To build images
Change	Cannot be changed	Can be changed (temporary)	Can be edited
Life Cycle	Permanent until deleted	Temporary (can be stopped/removed)	Persistent
Size	Larger	Smaller (only required layers)	Very small
Example	mysql:8.0, nginx:latest	Running nginx container	Dockerfile

5. OVERVIEW DIAGRAM



6. DOCKER LIFECYCLE



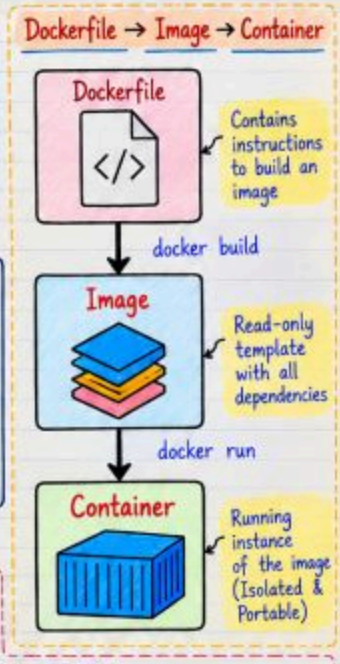
Key Takeaway : Build once → Run anywhere → Lightweight → Portable → Consistent

DOCKERFILE EXPLAINED

1. What is Dockerfile ?



- A Dockerfile is a **text file** that contains a set of instructions to build a Docker image automatically.
- It defines the environment, dependencies, configuration and commands to run the app.



2. Sample Dockerfile

```
1  FROM ubuntu:22.04                    # Base image
2  WORKDIR /app                         # Set working directory
3  COPY . /app                          # Copy files to image
4  RUN apt update && apt install -y python3 # Install python
5  RUN pip3 install flask               # Install flask
6  ENV FLASK_ENV=production             # Set environment
7  EXPOSE 5000                          # Expose port
8  CMD ["python3", "app.py"]            # Default command
```

3. Important Instructions

FROM	: Specifies the base image.
WORKDIR	: Sets the working directory inside the container.
COPY	: Copies files or directories from host to container.
ADD	: Similar to COPY, also supports URLs and auto-extracts .tar files.
RUN	: Executes commands during image build (e.g. install packages).
ENV	: Sets environment variables inside the container.
EXPOSE	: Informs Docker that the container listens on the given port.
CMD	: Specifies the default command to run when container starts.
ENTRYPOINT	: Configures a container that will run as an executable.

4. CMD vs ENTRYPOINT

Feature	CMD	ENTRYPOINT
Purpose	Provides default command and arguments	Configures the container to run as executable
Can be overridden	Yes (can be overridden)	No (cannot be overridden easily)
Instruction limit	Only one CMD allowed	Only one ENTRYPOINT allowed
Use case	Default behaviour	When container must always run a specific program
Example	CMD ["python3", "app.py"]	ENTRYPOINT ["python3", "app.py"]

5. Key Takeaways



- ☑ Dockerfile builds the image.
- ☑ Image is a read-only blueprint.
- ☑ Container is the running instance.
- ☑ Build once, run anywhere!



DOCKER NETWORKING

Docker containers can communicate with each other and with external world using different types of networks.

1 Bridge (Default)

- Containers on the same bridge network can communicate with each other.
- Default network created by Docker.
- Suitable for most use cases.

2 Host

- Container shares the host network.
- No network isolation (container uses host's network).
- Best performance.

3 None

- No network access.
- Useful for complete isolation.

4 Overlay

- Used to connect containers across multiple Docker hosts (Swarm mode).
- Useful for multi-host applications.

5 Storage in Docker

- Docker provides storage options to persist data generated by containers.

6 Volumes

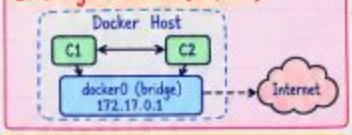
- Managed by Docker.
- Stored in Docker host (/var/lib/docker/volumes).
- Recommended for production.

7 Bind Mounts

- Uses a directory from the host.
- Platform dependent.
- Good for development (code changes reflect immediately).

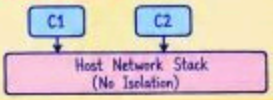
Network Types Diagrams

1. Bridge Network (Default)



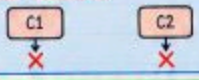
→ Containers communicate with each other and access to external network.

2. Host Network



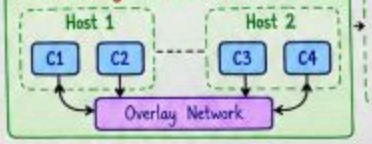
→ Containers share the host network. Best performance.

3. None Network



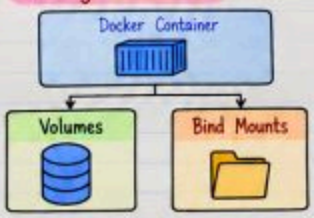
→ No network access. Complete isolation.

4. Overlay Network (Swarm)



→ Connect containers across multiple hosts. Requires Docker Swarm.

Storage in Docker



- Data persists even if container is removed.
- Volumes are managed by Docker.
- Bind Mounts use host directories.

Volumes vs Bind Mounts

Feature	Volumes	Bind Mounts
Managed by	Docker	Host
Location	Inside Docker host (/var/lib/docker/volumes)	Anywhere on host system
Portability	High	Lower (depends on host path)
Backup	Easy (docker volume backup)	Manual
Best for	Production	Development
Performance	Better (Docker optimized)	Good

Key Takeaway

- ☑ Use Volumes for production and persistent data.
- ☑ Use Bind Mounts for development.
- ☑ Choose the right network based on isolation and communication needs.

DOCKER COMPOSE

Docker Compose is a tool for defining and running multi-container Docker applications.

1. What is Docker Compose ?

- Docker Compose allows us to define and run multi-container Docker applications.
- It uses a YAML file to configure services, networks and volumes.
- With a single command, we can start, stop and manage entire application.

YAML

2. docker-compose.yml (Example)

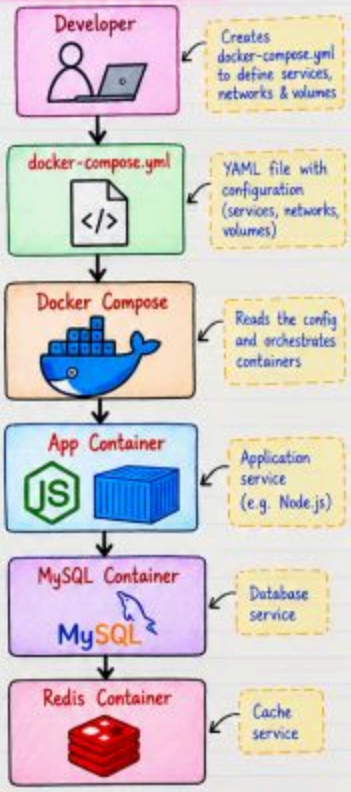
```yaml
version: '3.8'                    # Compose file version
services:                         # All services
  app:                            # Application service
    build: ./app                  # Build from ./app
    container_name: myapp         # Name of container
    ports:
      - "8080:8080"               # Port mapping (host:container)
    depends_on:
      - db                        # Depends on db service
    environment:
      - DB_HOST=db                # Environment variable

  db:                             # Database service
    image: mysql:8                # Use MySQL image
    container_name: mysql         # Name of container
    environment:
      - MYSQL_ROOT_PASSWORD=root  # Set root password
```

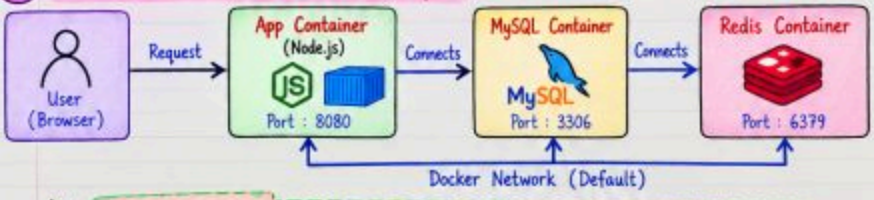
3. Important Commands

Command	Description
docker compose up	Build (if needed) and start all services
docker compose down	Stop and remove containers, networks, volumes
docker compose logs	View logs of all services
docker compose ps	List all running services
docker compose build	Build or rebuild services

4. How it Works ?



5. Multi Container Architecture (Example)



Key Takeaways

- ☑ Docker Compose simplifies multi-container applications.
- ☑ Uses a single YAML file for configuration.
- ☑ Manages services, networks and volumes.
- ☑ Build once, run everything with a single command.

Docker Container Lifecycle

A Docker container goes through different states during its lifetime.

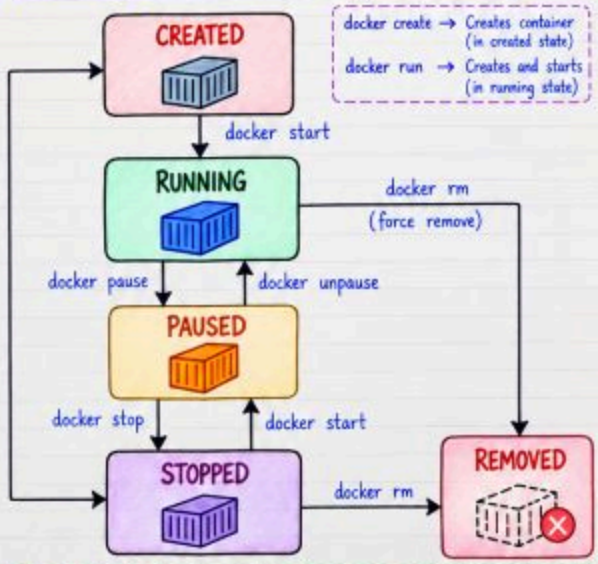
1 Created
- Container is created but not started.
- It has a filesystem, configuration and metadata.

2 Running
- Container is up and running.
- Main process (PID 1) is executing.

3 Paused
- Container is running but all processes are paused.
- Resources are frozen (CPU, memory etc).

4 Stopped
- Container is stopped but still exists.
- It can be started again.

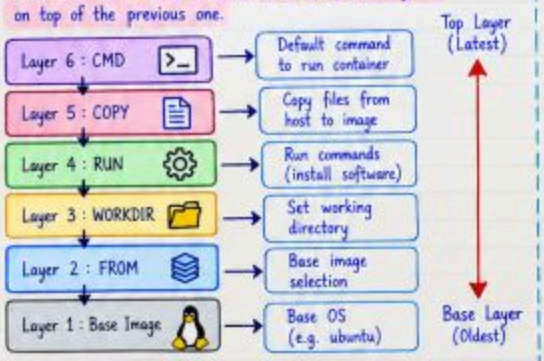
5 Removed
- Container is deleted and cannot be recovered.
- All data inside the container is lost (except volumes).



Remember :
- docker stop → stops the container.
- docker start → starts the stopped container.
- docker rm → removes the container.
- docker rm -f → force removes the container.

Docker Image Layers

Docker images are built using a layered architecture. Each instruction in the Dockerfile adds a new layer on top of the previous one.



Layer Caching

Docker caches each layer. If a layer does not change, it is reused from cache during build.

Layer 6	✗ Rebuilt (changes here)
Layer 5	✓ Cached (reused)
Layer 4	✓ Cached (reused)
Layer 3	✓ Cached (reused)
Layer 2	✓ Cached (reused)
Layer 1	✓ Cached (reused)

If a layer changes, all layers above it are rebuilt. Layers below it are reused from cache.

Key Takeaways :

- ☑ Understand container states and commands.
- ☑ Docker images are built in layers.
- ☑ Cache makes Docker builds faster.
- ☑ Use smaller layers for better performance.

DOCKER COMMANDS



Most commonly used Docker commands with examples

S.No	COMMAND	DESCRIPTION	EXAMPLE
1	docker version	Show Docker version information	docker version
2	docker info	Display system-wide information	docker info
3	docker login	Log in to Docker Hub	docker login
4	docker logout	Log out from Docker Hub	docker logout
5	docker search	Search images on Docker Hub	docker search nginx
6	docker pull	Pull an image from Docker Hub	docker pull nginx
7	docker images	List all local Docker images	docker images
8	docker build	Build an image from a Dockerfile	docker build -t myapp .
9	docker run	Run a container from an image	docker run -d --name myctr nginx
10	docker ps	List running containers	docker ps
11	docker stop	Stop a running container	docker stop myctr
12	docker start	Start a stopped container	docker start myctr
13	docker restart	Restart a container	docker restart myctr
14	docker rm	Remove a container	docker rm myctr
15	docker logs	View logs of a container	docker logs myctr
16	docker exec	Execute a command in a container	docker exec -it myctr bash
17	docker inspect	Inspect a container or image	docker inspect myctr
18	docker cp	Copy files/folders between container and host	docker cp myctr:/app ./app
19	docker network ls	List all Docker networks	docker network ls
20	docker volume ls	List all Docker volumes	docker volume ls
21	docker compose up	Start services defined in docker-compose.yml	docker compose up -d
22	docker compose down	Stop and remove services, containers, networks.	docker compose down
22	docker compose logs	View logs from services	docker compose logs -f
22	docker compose ps	List services defined in compose file	docker compose ps

Quick Tips

- ☑ Use -d to run containers in detached mode (background).
- ☑ Use docker --help to see all available commands.
- ☑ Use docker <command> --help for command specific help.
- ☑ Use docker ps -a to list all containers (running + stopped).
- ☑ Use docker images -a to list all images.

Note:

Replace myctr, myapp, nginx with your own container or image name.



Answer Like a Pro!

DOCKER INTERVIEW QUESTIONS

1 What is Docker ?

Ans:
Docker is an open-source platform that allows developers to build, package and run applications in containers. It ensures consistency across different environments (development, testing, production).

docker

2 Difference between Image and Container ?

Ans:
- **Image:** Read-only template with instructions to create a container.
- **Container:** Running instance of an image. It is writable and has a lifecycle.

Image → Container

3 CMD vs ENTRYPOINT ?

Ans:
- **CMD:** Provides default command and arguments. Can be overridden.
- **ENTRYPOINT:** Configures a container that will always run as an executable. CMD can be passed as arguments to ENTRYPOINT.

ENTRYPOINT + CMD

4 COPY vs ADD ?

Ans:
- **COPY:** Copies files/directories from host to container.
- **ADD:** Same as COPY, but also supports URLs and automatically extracts tar files.

COPY / ADD

5 Volume vs Bind Mount ?

Ans:
- **Volume:** Managed by Docker, stored in Docker host (/var/lib/docker/volumes). Recommended for production.
- **Bind Mount:** Maps a file or directory from the host to the container. Good for development (code changes reflect immediately).

Volume: /var/lib/docker/volumes

Bind Mount: Host Directory ⟷ Container

6 Bridge vs Host Network ?

Ans:
- **Bridge:** Default network. Containers communicate via Docker bridge (NAT). Suitable for most use cases.
- **Host:** Container shares the host's network namespace. No isolation.

Bridge Network: docker0 (NAT) → Internet

Host Network: Host OS Network

7 What is Docker Compose ?

Ans:
Docker Compose is a tool for defining and running multi-container Docker applications using a docker-compose.yml file. It helps to configure services, networks and volumes with a single command.

docker-compose.yml

- ☑ Services
- ☑ Networks
- ☑ Volumes

8 What are Dockerfile Layers ?

Ans:
Each instruction in a Dockerfile creates a new layer on top of the previous one. Layers are cached and reused to make builds faster and smaller.

Layer 4 / Layer 3 / Layer 2 / Layer 1 (Latest → Base)

9 What is Docker Hub ?

Ans:
Docker Hub is a cloud-based container registry service provided by Docker to store, share and distribute Docker images publicly or privately.

docker hub

- ☑ Store images
- ☑ Share images
- ☑ Public / Private registry

10 What is Docker Registry ?

Ans:
A Docker Registry stores Docker images. Docker Hub is a public registry. You can also run a private registry (e.g., Harbor, Nexus, or self-hosted).

- ☑ Store images
- ☑ Versioned images
- ☑ Public / Private

11 Docker Best Practices ?

Ans:
- ☑ Use official base images.
- ☑ Keep images small (use .dockerignore).
- ☑ Don't run containers as root.
- ☑ Use specific image tags (avoid :latest).
- ☑ Use multi-stage builds.
- ☑ Clean up unused images, containers, volumes regularly.
- ☑ Use Docker secrets for sensitive data (not in Dockerfile).
- ☑ Scan images for vulnerabilities (e.g., Trivy, Snyk).
- ☑ Use health checks to monitor containers.
- ☑ Use volumes for persistent data.
- ☑ Follow least privilege and security best practices.

KEY TAKEAWAY
Build Secure, Small & Reliable Containers!